Dayanıklılık Mühendisliği: Chaos Monkey ve Gizli Bağımlılıklar

ÖFY

October 2026

Özet

Bir sistemin ya da kurumun taşıdığı teknik borcun ne olduğu okuyarak öğrenilebilir; ancak bu borcun kararları nasıl bozduğu ve küçük değişiklikleri neden günlerce uzattığı çoğunlukla yaşanarak öğrenilir. Bunun nedeni, sistemi ayakta tutan bilginin büyük bölümünün yazıya dökülmemiş, adı konmamış örtük bilgi olmasıdır. Küçük ve güvenli bir sapma, bu örtük bilgiyi konuşmaya zorlar: normal akışta sessizce işleyen kural, sapma bir sürtünme ürettiğinde açıklanmak zorunda kalır ve böylece adlandırılmış, paylaşılabilir bir bilgiye dönüşür. Netflix'in Chaos Monkey aracı bu mantığın yazılımdaki otomatik biçimidir; çalışan sistemde rastgele sunucu kapatarak hata toleransını varsaymak yerine kanıtlar ve belgelenmemiş bağımlılıkları görünür kılar. Tersine mühendislik de aynı döngüye dayanır: sapma üretilir, sonuç gözlenir, bir çıkarsama yapılır ve bu çıkarsama yeni bir deneyle sınanır. Bu döngü bir sistemin davranışını ve çalışma sınırlarını ortaya çıkarır, ancak iç yapısını tekil olarak belirlemez; aynı dış davranışı farklı malzemeler, geometriler ve üretim süreçleri verebilir. Bu nedenle tersine mühendislik bir kopyalama yöntemi değil, hipotez eleme sürecidir ve yetkinliğin ölçüsü söküp inceleme becerisi değil, kalifikasyon, benzetim, eskime yönetimi ve yeniden tasarım gibi doğrulama işleridir. Türk savunma sanayiinde parça ikamesi ve alt sistem düzeyinde kurumsallaşmış bir kabiliyet bulunduğuna dair kamuya açık işaretler vardır; jet motoru gibi en ileri sistemlerde ise dünyada yalnızca birkaç ekosistem bağımsız bilgi üretebilmektedir.

Giriş

Karmaşık bir sistemin nasıl çalıştığı çoğu zaman tam olarak bilinmez. Bu durum iki biçimde ortaya çıkar. Birincisinde sistem kurumun kendi ürünüdür, ancak yıllar içinde büyümüş ve parçaları arasındaki ilişkiler belgelenmemiştir. İkincisinde sistem başkası tarafından üretilmiştir ve teknik dokümanı ya hiç yoktur ya da eksiktir. Her iki durumda da problem aynıdır: sistemin iç yapısı, dışarıdan gözlenen davranışından yola çıkarak anlaşılmalıdır.

Bu problemin bir de insan tarafı vardır. Bir yazılım geliştiricinin iyi kod yazabilmesi için çok hata yapması, çok test etmesi ve başkasının yazdığı kodu anlamaya uğraşması gerekir. Bu deneyimlerin hepsi zahmetlidir. Bu zahmeti yaşamamış birinin, yalnızca okuyarak bir kod tabanının ya da bir kurumun birikmiş sorunlarını kavraması beklenemez. Buradan çıkan soru şudur: bir yapıyı anlamak için o zahmet aynen yaşanmak zorunda mıdır, yoksa aynı bilgi daha kısa ve güvenli bir yoldan elde edilebilir mi.

Notun merkezî terimleri şunlardır. **Teknik borç** (*technical debt*), bir yazılımda hızlı ilerlemek için alınan kestirme kararların sonradan her değişikliği yavaşlatan birikmiş maliyetidir; terim Cunningham'a aittir (Cunningham, 1992). **Örtük bilgi** (*tacit knowledge*), uygulayan kişinin bildiği ama sözle tam olarak aktaramadığı bilgidir (Polanyi, 1966). **Tersine mühendislik** (*reverse engineering*), bir sistemin bileşenlerini ve bu bileşenler arasındaki ilişkileri belirlemek ve sistemi daha yüksek bir soyutlama düzeyinde yeniden ifade etmek için yapılan inceleme sürecidir (Chikofsky ve Cross, 1990). **Kaos mühendisliği** (*chaos engineering*), bir sistemin üretim ortamındaki çalkantılı koşullara dayanabildiğine dair güven oluşturmak amacıyla sistem üzerinde deney yapma disiplinidir (Basiri vd., 2016).

Yaygın kabul, tersine mühendisliğin bir ürünü söküp kopyalamak olduğu yönündedir. Bu kabul eksiktir. Bir parçanın geometrisini ölçmek, o parçanın hangi malzemeden, hangi üretim süreciyle ve hangi toleranslarla yapıldığını söylemez. Asıl yetkinlik, işlevi anlamak, eşdeğer bir çözüm üretmek ve bu çözümün gerçekten çalıştığını kanıtlamaktır.

Not, yazılım tarafında Chaos Monkey örneğini, donanım ve savunma tarafında tersine mühendisliğin kurumsal bileşenlerini kapsar. Belirli şirketlerin gizli kabiliyetleri, tersine mühendisliğin hukuki boyutu ve herhangi bir askerî sistemin kopyalanmasına dönük teknik ayrıntı kapsam dışındadır.

Deneyim ve Örtük Bilgi

Okuyarak öğrenilen ile yaşayarak öğrenilen

Teknik borç hakkında bir metin okuyan kişi borcun var olduğunu öğrenir. Öğrenemediği şey borcun etkisidir: tek satırlık bir değişikliğin neden üç gün sürdüğü, ekibin belirli bir dosyaya neden dokunmak istemediği, hangi kestirmenin hangi gece kesintisine yol açtığı. Bu bilgi belgede yazmaz; çünkü onu bilenler de çoğu zaman cümleye dökemez.

Bu ayrımı açıklayan dört yerleşik kavram vardır:

- **Örtük bilgi.** Polanyi'nin formülüyle, insan söyleyebildiğinden fazlasını bilir (Polanyi, 1966). Deneyimli bir geliştirici bir kod parçasının "kötü koktuğunu" hisseder ama nedenini hemen sıralayamaz.
- **Durumsal öğrenme** (*situated learning*). Bilgi, kullanıldığı gerçek bağlamın ve uygulayıcı topluluğunun içinde edinilir; bağlamdan koparıldığında büyük kısmı aktarılamaz (Lave ve Wenger, 1991).
- **Deneyimsel öğrenme** (*experiential learning*). Öğrenme, deneme, sonuç görme, üzerine düşünme ve yeniden deneme döngüsüyle gerçekleşir (Kolb, 1984).
- **Empati açığı** (*empathy gap*). Bir durumu o anda yaşamayan kişi, o durumun davranış üzerindeki etkisini sistematik olarak küçümser (Loewenstein, 1996). Bakım yükünü taşımamış bir yönetici, o yükün maliyetini düşük tahmin eder.

Yazılı belgenin yetmediğine dair kanıt

Collins, 1970'lerde bir lazer türünün laboratuvarlar arasında nasıl yayıldığını incelemiştir. Bulgusu şudur: yalnızca yayımlanmış makalelerden yola çıkan hiçbir laboratuvar çalışan bir lazer kuramamış; kurabilenlerin hepsi, daha önce kurmuş bir ekiple doğrudan temas etmiştir (Collins, 1974). Yazılı belge eksiksiz göründüğü hâlde, cihazı çalıştıran bilginin bir kısmı belgede yer almamıştır.

Bu bulgu notun başındaki sezgiyi destekler, ancak bir düzeltme de getirir. Zahmetin aynen yaşanması şart değildir; şart olan, sistemle doğrudan temastır. Kayıtlar, geçmiş değişiklikler, kırılan testler, ekiple görüşmeler ve küçük güvenli değişiklikler bu teması sağlar. Bu yol salt okumaktan farklıdır: sistem kontrollü biçimde denenmiş olur.

Küçük ve Güvenli Sapma

Sapmanın işlevi

Küçük ve güvenli sapma, bir sistemin tek bir koşulunun, zararı sınırlı ve geri alınabilir biçimde kasıtlı olarak değiştirilmesidir. Sapma sistemi bozmak için yapılmaz; sisteme bir soru sormak için yapılır: bu varsayım biraz değişirse sistem neyi korumaya çalışır.

Bir sistemin ne yaptığı normal koşulda görünür. Neden öyle yaptığı ise sınırına yaklaştığında görünür. Sınıra yakın verilen tepki, mimarideki öncelikleri, gizli bağımlılıkları ve güvenlik katmanlarını açığa çıkarır. Bu notta bu yaklaşım için *sınırdan anlama* çalışma adı kullanılmaktadır; yerleşik bir terim değildir, mühendislikteki arıza modu düşüncesine (*failure-mode thinking*) yakındır.

Yüksek riskli sistemlerde sapma gerçek sistem üzerinde yapılmaz. Bir hava savunma sistemi ya da uçuş hâlindeki bir uçak deney alanı değildir. Aynı düşünme biçimi benzetim modeli, alt bileşen, test düzeneği ve geçmiş arıza kayıtları üzerinde uygulanır.

Adsız bilginin adlandırılması

Normal akışta örtük bilgi sessizce çalışır. “Bunu böyle yapıyoruz” denir, ama nedeni dile getirilmez; çünkü kimse sormamıştır. Küçük bir sapma bu sessizliği bozar. Sapma bir sürtünme ürettiğinde deneyimli kişi “o parça öyle değişmez, çünkü...” diye başlayan bir cümle kurmak zorunda kalır. O “çünkü”, örtük bilginin söze dökülmüş hâlidir.

Dönüşüm şu adımlardan oluşur:

1. Adı konmamış bir pratik sorunsuz işlemektedir.
2. Küçük ve güvenli bir sapma yapılır.
3. Sonuç değişir; bir sürtünme ya da beklenmeyen bir davranış ortaya çıkar.
4. “Neden böyle oldu” sorusu sorulur.
5. Deneyimli kişi ya da sistemin kendi davranışı bir açıklama verir.
6. Açıklama bir kural olarak adlandırılır, yazılır ve test edilebilir hâle gelir.

Üçüncü adımda sonuç değişmezse kural henüz görünmemiştir; başka bir sapma denenir. Sapma dönüşümün tetikleyicisi, adlandırılmış kural ise çıktısıdır.

İki örnek

Espresso makinesi

Bir kafede deneyimli barista, bir espresso çıkardıktan sonra makineyi hemen yeniden çalıştırmaz. Herkes buna uyar, ama kimse nedenini söylemez. Yeni başlayan biri beklemeden ikinci kahveyi alır ve kahve acı çıkar. Barista o zaman

açıklar: ilk işlemden sonra basınç ve sıcaklığın dengelenmesi gerekir. Sapmadan önce bilgi "bekle" biçiminde bir alışkanlıktı; sapmadan sonra "makinenin basıncı ve sıcaklığı dengeye gelmelidir" biçiminde adı olan bir kurala dönüşmüştür.

Ödeme sistemi

Bir ekipte ödeme isteklerinin daha hızlı işlenmesi için paralel çalıştırılması önerilir. Güvenli bir test ortamında paralellik biraz artırılır ve bazı ödemelerin iki kez işlenebildiği görülür. Kıdemli bir geliştirici nedenini söyler: aynı isteğin ağ sorunları yüzünden tekrar gelmesi bu akışta olağandır; bu yüzden her işlem bir kimlikle işaretlenir ve ikinci kez çalışması engellenir. Bir işlemin birden çok kez uygulandığında tek kez uygulanmış gibi sonuç vermesine **idempotentlik** (*idempotency*) denir. Sapmadan önce bu, ekibin bildiği ama yazmadığı bir alışkanlıktı; sapmadan sonra "ödeme akışı idempotent olmalıdır" biçiminde test edilebilir bir kural olmuştur.

Yöntemin literatürdeki karşılıkları

Yöntemin tamamını karşılayan tek bir yerleşik ad yoktur. En yakın kavramlar şunlardır:

- **Bilgi açığa çıkarma** (*knowledge elicitation*). Uzmanın söze dökmediği bilgiyi soru, örnek ve gözlemle görünür hâle getirme çalışmalarının genel adıdır.
- **Dışsallaştırma** (*externalization*). Nonaka ve Takeuchi'nin bilgi yaratma modelinde örtük bilginin açık, paylaşılabilir bilgiye dönüştüğü aşamadır (Nonaka ve Takeuchi, 1995).
- **İhlal deneyi** (*breaching experiment*). Garfinkel, toplumsal hayatın yazılmamış kurallarını görünür kılmak için bu kuralları kasıtlı ve küçük ölçekte çiğnetmiş, ortaya çıkan tepkiden kuralın kendisini okumuştur (Garfinkel, 1967). Küçük sapma fikrinin sosyolojideki en yakın karşılığı budur; fark, Garfinkel'in toplumsal normları, bu notun ise teknik sistemleri ve ekip pratiklerini konu almasıdır.

Bu notta yöntemin bütünü için *sapmayla adlandırma* çalışma adı kullanılmaktadır: amaç hata bulmak değil, küçük bir fark yaratarak normalde sessiz kalan bilgiyi söze zorlamaktır.

Chaos Monkey

Tanım ve işleyiş

Chaos Monkey, Netflix'in bulut altyapısında çalışan bir servistir. Servis, aynı işi yapan sanal makine gruplarını bulur, bu gruplardan birinin içinden rastgele bir sanal makine seçer ve onu kapatır (Bennett ve Tseitlin, 2012). Ardından cevaplanan soru tektir: bir parça beklenmedik biçimde kaybolduğunda hizmet, kullanıcı fark etmeden çalışmaya devam ediyor mu.

Aracın amacı arıza çıkarmak değildir. Amaç, sistemin kendi kendine toparlanma yeteneğini doğrulamaktır. Kapatma işlemi mühendislerin görev başında olduğu saatlerde yapılır; böylece beklenmeyen bir sonuç ortaya çıktığında müdahale edecek kişiler hazırdır.

Hata toleransı

Hata toleransı (*fault tolerance*), bir sistemin bir ya da birkaç bileşeni arızalandığında hizmetini sürdürebilme özelliğidir. Bu özellik iki şeyle sağlanır: aynı işi yapan yedek bileşenler ve arızalanan bileşenin yükünü yedeğe otomatik aktaran bir mekanizma. Örneğin aynı servisin üç kopyası çalışıyorsa ve istekler bunlara dağıtılıyorsa, bir kopyanın kaybı kullanıcıya yansımaz. Chaos Monkey'in sınadığı özellik budur.

Adın kökeni

Netflix mühendisleri adı şu benzetmeyle açıklamıştır: veri merkezine salınmış, elinde silah olan vahşi bir maymun makineleri rastgele devre dışı bırakır ve kabloları kemirir; buna rağmen müşterilere hizmet kesintisiz sürer (Izrailevsky ve Tseitlin, 2011). Benzetmedeki maymun, öngörülemeyen arızayı temsil eder.

Netflix'in dayandığı ilkeler

Netflix, Chaos Monkey'i 2011'de bulut altyapısına geçiş döneminde tanıtmıştır. Yaklaşım dört ilkeye indirgenebilir:

- **Tek bir makineye güvenilmez.** Bulut ortamında herhangi bir sunucu herhangi bir anda kaybolabilir. Sistem, en zayıf halkasından daha dayanıklı olacak biçimde tasarlanır.
- **Arıza anında insan müdahalesi beklenmez.** Toparlanma otomatik olmalıdır; gece yarısı bir mühendisin uyanmasına bağlı bir sistem dayanıklı sayılmaz.
- **Hata toleransı varsayılmaz, düzenli olarak kanıtlanır.** Hataya dayanıklı bir mimari tasarlamak yeterli değildir. Netflix bunu patlak lastik örneğiyle anlatır: bagajda yedek lastik bulunması, o lastiğin şişirilmiş olduğunu ve değiştirmek için gereken aletlerin elde olduğunu göstermez (Izrailevsky ve Tseitlin, 2011).

- **Hatalar kullanıcıdan önce bulunur.** Arıza kontrollü biçimde ve hazırlıklı bir anda üretildiğinde, aynı arızanın hazırlıksız bir anda kendiliğinden ortaya çıkmasının maliyeti önlenir.

Gizli bağımlılıkların görünür kılınması

Bir servisin başka bir servise bağımlı olduğu çoğu zaman belgelerde yer almaz; bağımlılık ancak bağımlı olunan parça ortadan kalktığında fark edilir. Chaos Monkey bu fark edişi rastlantıya bırakmaz. Bir sunucu kapatıldığında beklenmedik bir servis de duruyorsa, o iki parça arasında bilinmeyen bir bağ olduğu ortaya çıkmış olur. Araç sürekli çalıştığı için ekipler bu bağı ortadan kaldırmak zorunda kalır; aksi hâlde aynı kesinti tekrar eder.

Chaos Monkey'in ana amacı adsız bilgiyi adlandırmak değil, dayanıklılığı sürekli doğrulamaktır. Sapmayla adlandırma burada bir yan üründür: her beklenmedik kesinti, o güne kadar kimsenin yazmadığı bir varsayımı ortaya çıkarır.

Chaos Monkey'in başarısının ardından Netflix, farklı arıza türlerini üreten bir araç ailesi kurmuş ve buna Simian Army adını vermiştir. Bu ailede tek bir sunucu yerine bütün bir erişilebilirlik bölgesini (*availability zone*) devre dışı bırakan ya da ağ gecikmesi üreten araçlar yer alır (Izrailevsky ve Tseitlin, 2011).

Uygulama sınırları ve yakın kavramlar

Kaos deneyleri üç koşulla yapılır: deneyin etkileyebileceği alan, yani etki yarıçapı (*blast radius*) küçük tutulur; geri dönüş planı hazırdır; sistemin durumu deney boyunca ölçülmektedir. Başlangıç için doğru yer üretim ortamı değil, izole bir test ortamıdır.

Aynı aileden üç kavram vardır. **Hata enjeksiyonu** (*fault injection*), bir bileşene kasıtlı olarak hata verilmesidir; bağımlı bir servisin yavaşlatılması ya da veritabanı bağlantısının kesilmiş gibi gösterilmesi buna örnektir. **Dayanıklılık testi** (*resilience testing*), sistemin bozulma sonrası toparlanmasının ölçülmesidir. **Tatbikat günü** (*game day*), ekibin önceden planlanmış bir arıza senaryosunu birlikte yaşayıp müdahale sürecini sınadığı çalışmadır.

Çıkarsama ve Deney Döngüsü

Çıkarsamanın tanımı

Çıkarsama (*inference*), gözlenen ipuçlarından gözlenmeyen bir sonuca akıl yürütmeyle ulaşmaktır. Basit bir örnek: sabah yerin ıslak olduğu gözlenir ve gece yağmur yağdığı sonucuna varılır.

Bu sonuç kesin bilgi değildir. Yer, biri hortumla yıkadığı için de ıslanmış olabilir. Gözlemi açıklayan en makul nedeni seçmeye dayanan bu akıl yürütme türü, mantık literatüründe Peirce'ün adlandırmasıyla abdüksiyon (*abduction*) olarak bilinir (Peirce, 1931–1958). Bu türün ayırt edici özelliği, ulaşılan sonucun bir varsayım olması ve başka gözlemlerle sınanması gerektiğidir.

Sistem incelemesinde çıkarsama

Aynı yapı sistem incelemesinde şu biçimi alır:

1. **Küçük sapma:** Sistemde tek bir değişken kontrollü biçimde değiştirilir.
2. **Gözlem:** Değişiklikten sonra sistemin yavaşladığı görülür.
3. **Çıkarsama:** Değiştirilen değişkenin, sistemin kritik bir bağımlılığıyla ilişkili olduğu varsayılır.
4. **Doğrulama:** Bu varsayım, güvenli bir ortamda yapılan yeni bir deneyle sınanır.

Chaos Monkey bu döngünün otomatikleştirilmiş hâlidir: sapma bir sunucunun kapatılmasıdır, gözlem hizmetin durumudur, çıkarsama ise ortaya çıkan gizli bağımlılıktır. Fiziksel bir ürünün tersine mühendisliğinde de aynı döngü işler; yalnızca sapma bir yük, bir sıcaklık ya da bir giriş sinyali olur.

Döngünün değeri dördüncü adımdadır. Doğrulanmamış bir çıkarsama yalnızca bir tahmindir; tahmin üzerine kurulan tasarım, ilk beklenmedik koşulda başarısız olur.

Davranıştan iç yapıya geçişin sınırı

Sınır testleri bir sistemin hangi koşulda hangi biçimde başarısız olduğunu gösterir. Bu bilgiyle sistemin güvenli çalışma zarfı çıkarılır. Örneğin bir motor parçasının belirli bir sıcaklık ve yükte çatladığı görülürse, o sınırın altında kalınması gerektiği sonucuna varılır.

Buradan "parçanın alaşımı ve döküm yöntemi şudur" sonucuna geçilemez. Aynı dış davranışı birden fazla malzeme, geometri, soğutma düzeni ve üretim süreci verebilir. Bu duruma sistem kuramında **tanımlanabilirlik** (*identifiability*) problemi denir: eldeki gözlemler iç parametreleri tekil olarak belirlemeye yetmeyebilir (Bellman ve Åström, 1970). Sonuçtan nedene doğru giden bu tür problemlere genel olarak **ters problem** (*inverse problem*) adı verilir.

Bu yüzden tersine mühendislik bir *hipotez eleme* sürecidir. Her sapma ve gözlem, olası iç yapıların kümesini biraz daraltır; küme hiçbir zaman tek elemana inmeyebilir. Belirsizliği azaltan şey tek bir sert test değil, farklı türden gözlemlerin birbirini doğrulamasıdır.

Yapıyı Anlama Yöntemleri

Küçük sapma ve tersine mühendislik, bir yapıyı anlamak için kullanılan daha geniş bir araç ailesinin iki üyesidir. Bu araçlar tek bir disiplinin alt başlıkları değildir; mühendislik, bilim felsefesi, kontrol kuramı, tasarım ve kurumsal öğrenmenin kesişiminde durur. Ortak üst başlık *karmaşık sistemleri anlama yöntemleri* olarak adlandırılabilir. Araçlar dört kümede toplanır: gözlem, müdahale, açıklama ve sınır koyma.

Gözlem

Sistem değiştirilmeden izlenir.

Kavram	İçerik
Doğrudan gözlem	Davranış, tekrarlar ve istisnalar kaydedilir.
Etnografik gözlem	İnsanların söylediği değil, işi fiilen nasıl yaptığı izlenir.
Köken takibi	Bir unsurun nereden geldiği ve nasıl değiştiği izlenir.
Yol bağımlılığı	Yapının en iyi olduğu için değil, geçmişte seçilen yol nedeniyle böyle kaldığı görülür.
Ölçümleme	Sezgi yerine iz bırakan ölçüm noktaları konur.
Gözlenebilirlik	İç durumun dış belirtilerden ne kadar okunabildiği sorulur.
İstisna avcılığı	Genel kurala uymayan örnekler toplanır.
Negatif alan analizi	Sistemde bulunmayan ya da özellikle engellenen şey incelenir.
Karşılaştırmalı analiz	Benzer iki yapı kıyaslanarak ayırt edici ilke bulunur.
Varyans analizi	Aynı girdinin neden farklı sonuç verdiği araştırılır.

Müdahale

Sisteme ya da modeline dokunulur ve tepkisi incelenir.

Kavram	İçerik
Küçük ve güvenli sapma	Tek bir koşul sınırlı ve geri alınabilir biçimde değiştirilir.
Hata enjeksiyonu	Bir bileşene kasıtlı hata verilir.
Hassasiyet analizi	Hangi küçük değişikliğin sonucu en çok etkilediği bulunur.
Kenar durum analizi	Nadir ve uç koşullardaki davranış incelenir.
Minimum müdahale	En az değişiklikle en çok bilgi veren deney aranır.
Aktif öğrenme	Rastgele veri yerine en öğretici deney seçilir.
Benzetim	Deney gerçek sistem yerine modeli üzerinde yapılır.
Düşünce deneyi	Değişiklik yalnızca zihindeki modelde çalıştırılır.
Karşıolgusal düşünme	Bir parça olmasaydı neyin değişeceği sorulur.
Karşıt tasarım	Yapı ters bir ilkeyle kurulsaydı ne olacağı düşünülür.

Açıklama

Gözlenen davranışa bir neden ve bir model bulunur.

Kavram	İçerik
Abdüksiyon	Gözlemi en iyi açıklayan gizli neden seçilir.
Hipotez testi	"Neden buysa şu koşulda şu olmalı" öngörüsü sınanır.
Yanlışlama	Hipotez doğrulanmaya değil çürütülmeye çalışılır (Popper, 1959).
Mekanizma açıklaması	Neyin neyle birlikte olduğu değil, hangi süreçle olduğu bulunur.
Ayrıştırma	Yapı alt bileşenlere bölünür, her birinin rolü belirlenir.
Bağımlılık haritalama	Hangi unsurun hangisini etkilediği çıkarılır.
Arayüz analizi	Bileşenlerin birbirine hangi varsayımlarla bağlandığı incelenir.
Değişmez bulma	Ne değişirse değişsin sabit kalan kural aranır.
Soyutlama	Ayrıntı geçici olarak atılır, temel model çıkarılır.
Katman analizi	Yapı fiziksel, teknik, sosyal ve ekonomik katmanlarda ayrı okunur.
Geri besleme analizi	Bir etkinin dönüp sistemi nasıl yeniden etkilediği izlenir.
Gerilim analizi	Sistemin hangi hedefler arasında denge kurduğu bulunur.
Birinci ilkeler	Temel fiziksel ya da mantıksal sınırlara inilir.
Bayesçi güncelleme	Her gözlemle hipotezlerin ağırlığı yeniden ayarlanır.
Üçgenleme	Aynı sonuç farklı kanıt türleriyle doğrulanır.

Sınır koyma

Bilinenin ve bilinebilenin sınırı çizilir.

Kavram	İçerik
Tanımlanabilirlik	Gözlemlerin iç yapıyı tekil olarak belirleyip belirlemediği sorulur.
Kısıt analizi	Sistemin ne yapamadığı incelenir.
Dar boğaz analizi	Gerçek kapasiteyi belirleyen tek nokta bulunur.
Eşik etkisi	Küçük değişikliğin hangi noktadan sonra büyük sonuç verdiği aranır.
Ölçek analizi	Kuralın hangi ölçekte geçerli olduğu ayrılır.
Arıza modu düşüncesi	Sistemin nasıl bozulabileceği önceden sıralanır.
Kalıntı analizi	Modelin açıklayamadığı farkın ne söylediğine bakılır.
Model sınırı analizi	Açıklamanın nerede işe yaramaz hâle geldiği belirlenir.
Belirsizlik nicemleme	Bilinen ile bilinmeyen açıkça ayrılır.
Refleksivite	Ölçümün ve müdahalenin yapıyı değiştirdiği hesaba katılır.

Çekirdek üçlü

Bu ailede üç araç birbirini tamamlar. Küçük sapma veri üretir, abdüksiyon bu veriye olası bir açıklama kurar, değişmez bulma ise açıklamanın gerçekten temel bir kuralı yakalayıp yakalamadığını sınar. Chaos Monkey bu araçlardan altısını aynı anda kullanır: küçük sapma, arıza modu düşüncesi, gözlenebilirlik, bağımlılık haritalama, hipotez testi ve yanlışlama.

Tersine Mühendisliğin Kurumsal Bileşenleri

Tersine mühendislikte yetkin olmak, söküp incelemenin yanında ölçüm, malzeme bilimi, üretim disiplini, test altyapısı ve yeniden tasarlama kapasitesi gerektirir. Bu kapasite kurumlarda dört ayrı mühendislik işi olarak karşılık bulur.

Kavram	Cevapladığı soru
Kalifikasyon	Bu parça gereken şartlarda gerçekten çalışır mı?
Benzetim	Gerçek sistem riske atılmadan, şu koşulda ne olur?
Eskime yönetimi	Tedarikçi bu parçayı bırakırsa sistem nasıl ayakta kalır?
Yeniden tasarım	Aynı işlev güncel bileşenlerle nasıl yeniden kurulur?

Kalifikasyon

Kalifikasyon (*qualification*), bir parçanın ya da sistemin tanımlı çalışma şartlarını karşıladığının testle kanıtlanması ve belgelenmesidir. Örneğin bir elektronik kart sıcak, soğuk, titreşim ve nem altında test edilir; her testin sonucu kayda geçirilir. Kalifikasyon, "çalışıyor gibi görünüyor" ile "çalıştığı kanıtlandı" arasındaki farkı kapatır. Tersine mühendislikle üretilen eşdeğer bir parça, kalifiye edilmeden sisteme takılamaz; çünkü geometrik olarak aynı olan bir parça, malzeme ya da üretim farkı nedeniyle sahada farklı davranabilir.

Benzetim

Benzetim (*simulation*), gerçek sistemin dijital ya da fiziksel bir modelinin kurulması ve deneylerin bu model üzerinde yapılmasıdır. Örneğin bir motorun farklı irtifa ve sıcaklık koşullarındaki davranışı, motor gerçekten o koşullara sokulmadan model üzerinde incelenir. Benzetim iki nedenle gereklidir: gerçek sistem üzerinde deney pahalıdır ve bazı koşullar gerçek sistemde güvenli biçimde üretilemez.

Eskime yönetimi

Eskime yönetimi (*obsolescence management*), bir parçanın, yazılımın ya da malzemenin artık üretilmemesi durumunun önceden planlanmasıdır. Savunma sistemleri onlarca yıl kullanılır; içlerindeki elektronik bileşenlerin üretim ömrü ise çoğu zaman bundan çok daha kısadır. Bu nedenle bir sistemin ömrü boyunca bazı parçalarının piyasadan kalkması beklenen bir olaydır. Eskime yönetimi, hangi parçaların risk altında olduğunu izler ve ikame yolunu önceden hazırlar.

Yeniden tasarım

Yeniden tasarım (*redesign*), mevcut bir çözümün aynen kopyalanması yerine işlevinin korunarak yeni bir çözümün üretilmesidir. Örneğin artık bulunamayan bir elektronik modül, aynı işi yapan güncel bileşenlerle baştan geliştirilir.

Yeniden tasarım, tersine mühendisliğin kopyalamadan ayrıldığı noktadır: kopyalama özgün ürünün biçimini, yeniden tasarım ise işlevini hedef alır.

Yetkinliğin Ülkelere Dağılımı

Tersine mühendislik yetkinliği için tek bir ülke sıralaması yoktur; güçlü olunan alan ülkeden ülkeye değişir. Aşağıdaki eşleştirme, sanayi yapılarına dair genel kabule dayanan kaba bir resimdir ve ölçülmüş bir sıralama olarak okunmamalıdır.

Ülke	Öne çıkan alan
Japonya	Hassas üretim, malzeme, kalite kültürü, süreç disiplini
Almanya	Makine, otomotiv, endüstriyel sistemler, ölçüm mühendisliği
ABD	Karmaşık sistem mimarisi, havacılık, yazılım, araştırmadan ürüne geçiş
Güney Kore, Tayvan	Elektronik, yarı iletken üretimi, yüksek hacimde süreç iyileştirme
Hollanda	Aşırı hassas makine ve optik sistemler
Çin	Büyük ölçekli üretim, hızlı yineleme, yerelleştirme
İsviçre, İsveç, Singapur	Hassas teknoloji, araştırma ile üretim arasındaki bağ

Bu ülkelerin ortak özelliği bir millî nitelik değil, kurumsal bir alışkanlıktır: başarısızlık ölçülür, sınanır, kayda geçirilir ve öğrenilen bilgi yeniden üretime sokulur. Chaos Monkey'in yazılımda yaptığı iş, bu kurumların üretimde yaptığı işin aynısıdır.

İkinci gözlem, ileri teknolojide bilginin tek bir ülkede toplanmadığıdır. Modern çip üretimi bunun en açık örneğidir: tasarım, üretim ekipmanı, malzeme ve imalat aşamaları farklı ülkelere ve şirketlere dağılmıştır ve hiçbir ülke zincirin tamamına tek başına hâkim değildir (OECD, 2019). Ülkelerin genel yenilik kapasitesi için WIPO'nun yıllık Küresel İnovasyon Endeksi karşılaştırmalı bir gösterge sunar; ancak bu endeks tersine mühendislik yetkinliğini doğrudan ölçmez.

Türk Savunma Sanayiinde Durum

Kamuya açık işaretler

Türk savunma sanayiinde gözlem, hipotez ve doğrulama döngüsünün kurumsal karşılığı bulunduğuna dair kamuya açık işaretler vardır. Savunma Sanayii Başkanlığı (SSB) çatısı altında Türk Savunma Sanayii Ömür Devri Yönetimi Platformu (TSSÖDYP) kurulmuştur. Platform, bir sistemin ihtiyaç belirlemeden envanterden çıkarmaya kadar bütün ömrünü tek bir bütün olarak ele alan rehber dokümanlar yayımlamaktadır (SSB, 2021).

İki kaynak şunları bildirmektedir. Birincisi, bir TSSÖDYP dokümanının tersine mühendisliği, teknik özelliklerin bilinmediği ya da eksik olduğu durumlarda çalışan bir örneğin derinlemesine incelenmesinden eşdeğer çözüm üretmeye uzanan bir yöntem olarak tarif ettiğidir. İkincisi, SSB'nin test ve sertifikasyon alanındaki strateji belgelerinde test planlama, prosedür, doğrulama, raporlama ve yerli kalifikasyon altyapısının merkezî yetkinlik olarak yer aldığıdır. Bu iki dokümanın tam adları ve ilgili bölümleri bu not yazılırken doğrulanamamıştır; arşivde kullanılmadan önce özgün metinlerden kontrol edilmelidir.

Üç yetkinlik düzeyi

"Tersine mühendislik biliniyor mu" sorusu tek bir cevapla karşılanamaz; çünkü yetkinlik üç ayrı düzeyde ortaya çıkar.

Bakım ve ikame

Bulunamayan bir parçanın anlaşılması ve eşdeğerinin üretilmesidir. Eskime yönetiminin doğrudan uygulamasıdır.

Alt sistem düzeyi

Elektronik, mekanik, yazılım ya da malzeme bileşenlerinin test edilmesi ve yeniden tasarlanmasıdır. Kalifikasyon ve benzetim altyapısı gerektirir.

En ileri sistem düzeyi

Bir jet motoru ya da ileri bir algılayıcı gibi sistemlerde yalnızca davranışın değil, malzeme, üretim ve tasarım bilgisinin de bağımsız olarak kurulmasıdır. Bu düzeyde belirleyici olan örtük üretim bilgisidir: bir parçanın nasıl yapılacağı çoğu zaman belgelerde değil, onu yıllarca üretmiş ekiplerin deneyiminde durur. Bu bilgi sökülen bir üründen okunamaz. MacKenzie ve Spinardi, nükleer silah tasarımı üzerine çalışmalarında aynı sonuca varmıştır: tasarım bilgisinin önemli bir kısmı yazılı değildir ve onu taşıyan ekipler dağıldığında kaybolabilir (MacKenzie ve Spinardi, 1995).

Değerlendirme

İlk iki düzeyde Türkiye'de kurumsallaşmış bir kabiliyet bulunduğu söylenebilir. Üçüncü düzey, dünyada da yalnızca birkaç ekosistemin uzun yıllar, çok büyük test altyapısı ve birikmiş üretim deneyimiyle ulaşabildiği bir seviyedir. Kamuya açık bilgilerle belirli bir şirketin bu derinlikteki kapasitesi kesin olarak ölçülemez; savunma alanında asıl kabiliyet verisi gizlidir.

Sonuç

Deneyimin zahmeti ile küçük sapma aynı işi görür: ikisi de örtük bilgiyi yüzeye çıkarır. Fark maliyettedir. Yıllar süren bakım deneyimi bu bilgiyi yavaş ve rastlantıyla verir; planlı bir sapma aynı bilginin bir parçasını saatler içinde ve seçilmiş bir noktadan verir. Bundan çıkan pratik sonuç, bir ekibe yeni katılan kişinin belge okutularak değil, güvenli bir ortamda sistemi sınırına götürerek yetiştirilmesi gerektiğidir.

Chaos Monkey ile tersine mühendislik aynı yöntemin iki uygulamasıdır: bilinmeyen bir yapı, kontrollü müdahaleye verdiği tepkiden okunur. Belge sistemin nasıl tasarlandığını, deney ise gerçekte nasıl davrandığını gösterir; ikisi arasındaki fark gizli bağımlılıkların bulunduğu yerdir.

Yetkinliğin ölçüsü söküp inceleme becerisi değil, doğrulama altyapısıdır. Kalifikasyon laboratuvarı, benzetim modeli ve test kaydı tutma alışkanlığı bulunmayan bir kurum, ürünü ne kadar iyi sökerse söksün çıkarsamalarını sınayamaz. Bir ülkenin ya da şirketin bu alandaki seviyesi, test ve kalifikasyon altyapısına yaptığı yatırımdan okunabilir.

Çerçevenin üç sınırı vardır. Birincisi, sapma yalnızca birinin açıklayabildiği kuralları adlandırır; açıklayacak deneyimli kişi yoksa ya da sistemin tepkisi yorumlanamıyorsa sapma yalnızca bir arıza olarak kalır. İkincisi, Chaos Monkey sunucuların birbirinin yerine geçebildiği ve kapatılan bir örneğin dakikalar içinde yenisiyle değiştirilebildiği bulut ortamı için tasarlanmıştır. Fiziksel sistemlerde üretim ortamında kasıtlı arıza çıkarmak çoğu zaman mümkün değildir; uçuş hâlindeki bir motor rastgele kapatılamaz ve aynı işlev benzetim ile yer testlerine devredilir. Üçüncüsü, davranıştan iç yapıya geçişte tanımlanabilirlik sınırı kalkmaz; gözlem ne kadar artarsa artsın birden fazla iç yapı aynı davranışla uyumlu kalabilir. Ülke karşılaştırması da ölçülmüş verilere değil genel kabule dayanmaktadır.

Açık kalan soru, örtük üretim bilgisinin sapma yoluyla ne ölçüde elde edilebileceğidir. Sapmayla adlandırma, bilgiyi taşıyan biri ya da bir şey bulunduğunda çalışır: barista açıklar, kıdemli geliştirici açıklar, sistem tepki verir. Başkasının ürettiği bir üründe açıklayacak kimse yoktur; yalnızca ürünün tepkisi vardır. O tepkiden imalat bilgisinin çıkarılıp çıkarılamayacağı, yoksa bu bilginin yalnızca yıllar süren üretim deneyimiyle mi biriktiği, bu notun kurduğu ama cevaplamadığı sorudur. Collins'in ve MacKenzie ile Spinardi'nin bulguları ikinci cevabı destekler yöndedir.

Kaynaklar

- Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., Rosenthal, C. (2016). Chaos Engineering. *IEEE Software*, 33(3).
- Bellman, R., Åström, K. J. (1970). On Structural Identifiability. *Mathematical Biosciences*, 7.
- Bennett, C., Tseitlin, A. (2012). Chaos Monkey Released into the Wild. *Netflix Technology Blog*.

- Chikofsky, E. J., Cross, J. H. (1990). Reverse Engineering and Design Recovery: A Taxonomy. *IEEE Software*, 7(1).
- Collins, H. M. (1974). The TEA Set: Tacit Knowledge and Scientific Networks. *Science Studies*, 4(2).
- Cunningham, W. (1992). The WyCash Portfolio Management System. *OOPSLA '92 Experience Report*.
- Garfinkel, H. (1967). *Studies in Ethnomethodology*. Prentice-Hall.
- Izrailevsky, Y., Tseitlin, A. (2011). The Netflix Simian Army. *Netflix Technology Blog*.
- Kolb, D. A. (1984). *Experiential Learning: Experience as the Source of Learning and Development*. Prentice-Hall.
- Lave, J., Wenger, E. (1991). *Situated Learning: Legitimate Peripheral Participation*. Cambridge University Press.
- Loewenstein, G. (1996). Out of Control: Visceral Influences on Behavior. *Organizational Behavior and Human Decision Processes*, 65(3).
- MacKenzie, D., Spinardi, G. (1995). Tacit Knowledge, Weapons Design, and the Uninvention of Nuclear Weapons. *American Journal of Sociology*, 101(1).
- Nonaka, I., Takeuchi, H. (1995). *The Knowledge-Creating Company*. Oxford University Press.
- OECD (2019). *Measuring Distortions in International Markets: The Semiconductor Value Chain*. OECD Trade Policy Papers.
- Peirce, C. S. (1931–1958). *Collected Papers of Charles Sanders Peirce*. Harvard University Press.
- Polanyi, M. (1966). *The Tacit Dimension*. Doubleday.
- Popper, K. (1959). *The Logic of Scientific Discovery*. Hutchinson.
- SSB (2021). Türk Savunma Sanayii Ömür Devri Yönetimi Platformu (TSSÖDYP) rehber dokümanları. T.C. Cumhurbaşkanlığı Savunma Sanayii Başkanlığı.
- WIPO. *Global Innovation Index*. World Intellectual Property Organization, yıllık yayın.